Universidade Federal do Ceará – Campus Quixadá
QXD0010 – Estruturas de Dados

Relatório
Lista Circular Duplamente Encadeada

Aluno: Vinicius Macedo Freitas
Matrícula: 585767 Turma: ED 01A

1 Visão geral do programa

O programa, escrito em C++, implementa uma lista de números inteiros usando uma lista circular duplamente encadeada com nó cabeça, encapsulada na classe `List`. Ele é controlado por comandos digitados no terminal, o que permite criar várias listas e testar todas as operações da classe sem alterar o código.

O código está dividido em quatro arquivos:

- `Node.h`: define o nó da lista, com um inteiro (`key`) e os ponteiros `anterior` e `proximo`.
- `List.h`: declaração da classe `List`.
- `List.cpp`: implementação de todas as operações da classe.
- `main.cpp`: interface por comandos no terminal.

2 Estrutura de dados

A classe possui dois atributos: `m_head`, ponteiro para o nó cabeça, e `m_size`, a quantidade de elementos da lista (sem contar o nó cabeça). O nó cabeça não guarda dado: ele serve apenas como ponto de referência fixo.

Com a lista vazia, o `proximo` e o `anterior` do nó cabeça apontam para ele mesmo. Com a lista não vazia, o `proximo` do último nó aponta para o nó cabeça, e o `anterior` do nó cabeça aponta para o último, formando o círculo.

Essa organização traz duas vantagens. Primeiro, nenhum ponteiro vale `nullptr`, então inserir e remover nós usa sempre o mesmo código, inclusive no início, no fim e com a lista vazia. Segundo, o último elemento é acessado direto por `m_head->anterior`, e a lista pode ser percorrida nos dois sentidos.

3 Operações da classe List

- `List()`: cria o nó cabeça apontando para si mesmo e zera `m_size`.
- `List(const List& lst)`: construtor de cópia; cria uma lista nova e insere com `push_back` cada valor de `lst`, produzindo uma cópia independente.
- `~List()`: libera todos os nós, inclusive o nó m_head.
- `void push_back(int key)`: insere `key` no final da lista.
- `int pop_back()`: remove o último elemento e retorna seu valor. Lança `out_of_range` se a lista estiver vazia.

- `void insertAfter(int key, int k)`: insere `key` após o k-ésimo nó ($1 \leq k \leq$ `size`). Lança `out_of_range` se k for inválido.
- `void remove(int key)`: remove a primeira ocorrência de `key`.
- `void removeAll(int key)`: remove todas as ocorrências de `key` em uma única passada, guardando o próximo nó antes de cada remoção para não perder o caminho.
- `void print()` e `void printReverse()`: imprimem os elementos em ordem normal e reversa.
- `bool empty()` e `int size()`: informam se a lista está vazia e o seu tamanho.
- `void clear()`: remove todos os elementos e mantém apenas o nó cabeça.
- `void concat(List& lst)`: liga o final da lista atual ao início de `lst` apenas religando ponteiros, sem criar nós. Ao final, `lst` fica com apenas o nó cabeça. Se `lst` estiver vazia, nada é feito.
- `List operator=(const List& lst)`: cria uma cópia de `lst` com o construtor de cópia e troca seu conteúdo com o da lista atual; a cópia temporária libera os nós antigos.
- `bool operator==(const List& lst)`: duas listas são iguais se têm o mesmo tamanho e o k-ésimo valor de uma é igual ao k-ésimo da outra.
- `bool operator!=(const List& lst)`: negação do `operator==`.

4 Interface com o usuário

O `main.cpp` mantém um `vector` de listas e lê comandos digitados no terminal, um por linha, até receber `exit`. Cada comando indica a lista pelo seu índice: a primeira lista criada é a de índice 0, a segunda é a de índice 1, e assim por diante. Cada comando lido é repetido na tela precedido de `$`.

Comando	**Descrição**
`create`	Cria uma nova lista vazia.
`size l`	Mostra o tamanho da lista `l`.
`empty l`	Mostra `true` ou `false`.
`push_back l a1 ... an`	Insere os valores no final da lista `l`.
`pop_back l`	Remove e mostra o último elemento de `l`.
`insertAfter l k v`	Insere `v` após o k-ésimo nó de `l`.
`remove l v`	Remove a primeira ocorrência de `v`.
`removeAll l v`	Remove todas as ocorrências de `v`.
`print l`	Imprime a lista `l`.
`printReverse l`	Imprime a lista `l` em ordem reversa.
`clear l`	Remove todos os elementos de `l`.
`concat l1 l2`	Concatena `l2` ao final de `l1`; `l2` fica vazia.
`assign l1 l2`	`l1` passa a ser uma cópia de `l2`.
`equal l1 l2`	Mostra o resultado de `l1 == l2`.
`notequal l1 l2`	Mostra o resultado de `l1 != l2`.
`show`	Imprime todas as listas criadas.
`exit`	Encerra o programa.

Índice de lista inexistente imprime `lista inexistente`, e comando desconhecido imprime `comando inexistente`.

6 Exemplo de execução

```
$create
$push_back 0 1 2 3 2 4
$print 0
[ 1 2 3 2 4 ]
$printReverse 0
[ 4 2 3 2 1 ]
$insertAfter 0 2 99
$print 0
[ 1 2 99 3 2 4 ]
$removeAll 0 2
$print 0
[ 1 99 3 4 ]
$exit
```